

GMX · SECURITY REVIEW

GMX Pyth Updates.

A security review of Pyth Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
GMX	Pyth Updates	Guardian	July 19 – July 22, 2024	July 29, 2024

§ 00

Contents.

23 pages — overview, confidence ranking, scope, 14 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 14 findings	P. 08
§ 05	Disclaimer	P. 22
§ 06	About Guardian	P. 23

§ 01

Overview.

Guardian reviewed Pyth Updates across a single review round. The final report includes 8 Medium, 6 Low findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review
CLIENT	
GMX	
PROJECT	
Pyth Updates	
PREPARED BY	
Guardian	
AUDIT WINDOW	
July 19 – July 22, 2024	
FINAL REPORT	
July 29, 2024	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
red circle Critical	0	0	0	0	0
orange circle High	0	0	0	0	0
yellow circle Medium	8	0	8	0	0
blue circle Low	6	0	6	0	0
grey circle Informational	0	0	0	0	0
All severities	14	0	14	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Pyth Updates.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Staleness Validation Inaccuracy	Logical Error	MEDIUM	ACKNOWLEDGED	p. 8
M-02	Pyth Confidence Intervals Are Not Considered	Logical Error	MEDIUM	ACKNOWLEDGED	p. 9
M-03	Deviation Truncation Prevents Spread	Logical Error	MEDIUM	ACKNOWLEDGED	p. 10
M-04	Pyth Prices Are Used Even When Disabled	Logical Error	MEDIUM	ACKNOWLEDGED	p. 11
M-05	Experimental Pragma Usage	Warning	MEDIUM	ACKNOWLEDGED	p. 12
M-06	Spread Taken Multiple Times	Logical Error	MEDIUM	ACKNOWLEDGED	p. 13
M-07	Cumulative Delta Does Not Get Compounded	Logical Error	MEDIUM	ACKNOWLEDGED	p. 14
M-08	block.number used on Arbitrum	Logical Error	MEDIUM	ACKNOWLEDGED	p. 15
L-01	Unused Code	Optimization	LOW	ACKNOWLEDGED	p. 16
L-02	Full Price Duration Range Cannot Be Used	Logical Error	LOW	ACKNOWLEDGED	p. 17
L-03	Misnamed Parameters	Documentation	LOW	ACKNOWLEDGED	p. 18
L-04	Lacking Configuration Validations	Documentation	LOW	ACKNOWLEDGED	p. 19
L-05	Lack of maxPriceUpdateDelay Validation	Warning	LOW	ACKNOWLEDGED	p. 20
L-06	Array Length Mismatch	Warning	LOW	ACKNOWLEDGED	p. 21

MEDIUM **M-01**

ACKNOWLEDGED

Staleness Validation Inaccuracy

CATEGORY Logical Error
LOCATION FastPriceFeed.sol: 340
REVIEW Main Review

DESCRIPTION

In the `_setPricesWithData` function the `_setLastUpdatedValues` call assigns the latest block details as the last update which is used to determine the staleness of a price.

However the Pyth price can be from up to 60 seconds before the current block, based on Pyths default configured staleness check. Therefore the `maxPriceUpdateDelay` and `priceDuration` validations will be off by up to 60 seconds.

RECOMMENDATION

Consider adjusting the `lastUpdatedAt` to keep track of the publish time for each token's price from Pyth. Otherwise adjust the `maxPriceUpdateDelay` and `priceDuration` to account for this potential 60 second lag time.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Pyth Confidence Intervals Are Not Considered

CATEGORY	Logical Error
LOCATION	FastPriceFeed.sol
REVIEW	Main Review

— DESCRIPTION

Pyth prices offer a confidence interval range for every price, this is because the exact price of an asset cannot be given due to difference across exchanges and reporting sources.

Pyth recommends that the confidence interval be used in two ways, which could be adapted to the use-case for the FastPriceFeed contract:

1. Opt to not use the Pyth price if the confidence range becomes too wide, as this is an indication of uncertain pricing which may lead to profitable arbitrage opportunities.
2. Apply the confidence range returned from Pyth as a spread, moving in the direction that would be disadvantageous for traders.

— RECOMMENDATION

Consider adopting one or both of these use-cases for the pyth confidence interval to provide protection for uncertain pricing results.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Deviation Truncation Prevents Spread

CATEGORY Logical Error
LOCATION FastPriceFeed.sol: 299
REVIEW Main Review

DESCRIPTION

In the `getPrice` function, the `diffBasisPoints` are calculated with round down precision loss which leads to `hasSpread` reporting false when the deviation has in fact exceeded the maximum allowable amount.

For example:

- `maxDeviationBasisPoints` = 100
- BTC reference price: 64,338
- BTC `fastPrice`: 64,987
- `diffBasisPoints` = $64,987 - 64,338 * 10000 / 64,338 = 100.873511766$, truncated to 100
- The `fastPrice` is considered to be within the `maxDeviation` because a strict greater than comparison is used

As a result prices which are above the `maxDeviationBasisPoints` can be flagged as within the deviation range and will not have a spread applied.

RECOMMENDATION

Use a `>=` comparison rather than a strict `>` comparison for the `maxDeviationBasisPoints` check.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Pyth Prices Are Used Even When Disabled

CATEGORY Logical Error
LOCATION FastPriceFeed.sol: 299
REVIEW Main Review

— DESCRIPTION

In the `getPrice` function a spread is used if the fastPrice is deemed to be unsatisfactory. In the event that the favorFastPrice function returns false a spread is applied where the fastPrice can be used if it is the worse price for traders.

However if the fastPrice has been flagged by watchers to be inaccurate or even malicious it should not be used in any capacity for the resulting price, even if this value is not desirable for traders.

— RECOMMENDATION

Consider handling the disableFastPriceVoteCount spread separately, where the spread does not include the fastPrice. Additionally consider if the maxCumulativeDeltaDiffs spread should use the fastPrice as well.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Experimental Pragma Usage

CATEGORY Warning
LOCATION FastPriceFeed.sol
REVIEW Main Review

DESCRIPTION

The `FastPriceFeed` contract now utilizes the experimental `ABIEncoderV2` to support the `bytes[] calldata` parameter for the `_setPricesWithData` function. There are several known bugs pertaining to the `ABIEncoderV2` in Solidity `0.6.12`.

Specifically, `AbiReencodingHeadOverflowWithStaticArrayCleanup` and `ABIDecodeTwoDimensionalArrayMemory` documented at <https://docs.soliditylang.org/en/v0.8.25/bugs.html> affect Solidity version `0.6.12`.

Neither of these issues affect the `FastPriceFeed` contract currently, however future changes to this contract may introduce unexpected vulnerable behavior due to these known issues.

Additionally, there may be further undocumented or undiscovered bugs affecting the `ABIEncoderV2` with Solidity version `0.6.12` which could affect the contracts in their present form.

RECOMMENDATION

Be aware of these known bugs when making future changes to the and consider upgrading to an up to date version of Solidity, where known `ABIEncoderV2` bugs have been addressed.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Spread Taken Multiple Times

CATEGORY Logical Error
LOCATION VaultPriceFeed.sol
REVIEW Main Review

DESCRIPTION

The FastPriceFeed's `getPrice` function is called by the VaultPriceFeed's `getPriceV2` function. In multiple cases, `getPrice` may return a price with a spread applied on top of it. However, the `getPriceV2` will apply another `_spreadBasisPoints` onto the returned price. Consequently, two spreads are applied on the price, exacerbating the impact on user's orders.

Furthermore, if `VaultPriceFeed:getPrice` is called, it will trigger function `getPriceV2` and then apply one more spread — `adjustmentBasisPoints`. Three spreads are applied to a single price.

RECOMMENDATION

Do not apply another spread in the VaultPriceFeed. Otherwise, if intended, clearly document this behavior.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Cumulative Delta Does Not Get Compounded

CATEGORY Logical Error
LOCATION FastPriceFeed.sol: 391-392
REVIEW Main Review

— DESCRIPTION

Sequential percent changes cannot be added to see the overall, cumulative price movement. Two 10% price movements is not 20% but 21%.

Consequently, the `cumulativeRefDelta` is less than the true move, leading to potentially not favoring the fast price, inaccurate event emission, and systems that rely on that data.

— RECOMMENDATION

To get the true measure of price movement, multiply the percent changes.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

block.number used on Arbitrum

CATEGORY	Logical Error
LOCATION	FastPriceFeed.sol: 424
REVIEW	Main Review

— DESCRIPTION

Using `block.number` on Arbitrum returns the L1 Mainnet block close to when the sequencer received the transaction. This may unnecessarily prevent updates with the revert "FastPriceFeed: minBlockInterval not yet passed".

— RECOMMENDATION

On Arbitrum, utilize ArbSys to fetch the Arbitrum block number.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Unused Code

CATEGORY Optimization
LOCATION FastPriceFeed.sol: 39
REVIEW Main Review

— DESCRIPTION

In the FastPriceFeed contract the BITMASK_32 variable is unused.

— RECOMMENDATION

Remove the BITMASK_32 variable.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Full Price Duration Range Cannot Be Used

CATEGORY Logical Error
LOCATION FastPriceFeed.sol: 355
REVIEW Main Review

DESCRIPTION

In the `_setPricesWithData` function the `pythPrice.publishTime` validation uses a strict greater than comparison with the `block.timestamp - _priceDuration` to determine if a price is stale.

However because a strict greater than is used the full `_priceDuration` is not available for prices retrieved from pyth.

RECOMMENDATION

Consider using a `>=` comparison to allow the final second in the fresh range to be used.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Misnamed Parameters

CATEGORY	Documentation
LOCATION	IPositionRouter.sol: 10
REVIEW	Main Review

— DESCRIPTION

In the `PositionRouter` Interface the first parameter for the `executeIncreasePositions` function is defined as the `_count`, however in the implementation for the `executeIncreasePositions` function this parameter is named `_endIndex`.

— RECOMMENDATION

Consider standardizing on one parameter name across the interface and function implementation.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Lacking Configuration Validations

CATEGORY	Documentation
LOCATION	FastPriceFeed.sol
REVIEW	Main Review

DESCRIPTION

In the `FastPriceFeed` contract there are lacking validations for the trusted setter functions, therefore if trusted addresses are compromised they can assign values which would cause stale prices or even DoS the exchange.

For instance, if the `maxPriceUpdateDelay` is assigned to 0 and the `spreadBasisPointsIfChainError` is assigned to the max `uint256`, calls to the `getPrice` function will revert and DoS all interactions with the GMX vault.

RECOMMENDATION

Consider adding validations that the values assigned in the trusted setter functions are within an expected range.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Lack of maxPriceUpdateDelay Validation

CATEGORY Warning
LOCATION FastPriceFeed.sol
REVIEW Main Review

DESCRIPTION

The `lastUpdatedAt` can only be updated every `minBlockInterval`. If `maxPriceUpdateDelay` is less than time passes within the `minBlockInterval`, function `getPrice` will always return the price with `spreadBasisPointsIfChainError` added.

RECOMMENDATION

Ensure `maxPriceUpdateDelay` is larger than the time passed within the `minBlockInterval`.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Array Length Mismatch

CATEGORY Warning
LOCATION FastPriceFeed.sol
REVIEW Main Review

DESCRIPTION

The length of `_tokens` is not validated against the length of `_maxCumulativeDeltaDiffs` in `setMaxCumulativeDeltaDiffs`.

This may lead to unintended consequences such as a token being set with the wrong max delta diff.

RECOMMENDATION

Consider validating that the two arrays are the same length.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.